实验报告：userprog-part1

课程名称：操作系统实践	**年级**：23 级	**上机实践成绩**：
指导教师：张民	**姓名**：叶晓良	
上机实践名称：userprog-part1	**学号**：10235101427	**上机实践日期**：2024/12/24
上机实践编号：	**组号**：	**上机实践时间**：2024/12/24

一、目的

本次实验所涉及并要求掌握的知识点：

1. 理解 Pintos 操作系统的功能流程及其内核的软件工程结构：从整体上把握操作系统的架构和设计原则。

2. 分析 Pintos 操作系统内核，识别并解决用户程序参数传递的问题：通过深入研究，学会如何诊断和修复操作系统中参数传递的缺陷。

3. 探究 Pintos 的中断处理机制，并学习操作系统中断功能的编写方法：增强对操作系统底层运作的理解，提升编程技能。

4. 完善 Pintos 的系统调用功能，使其能够处理用户中断请求：掌握如何扩展操作系统以满足特定需求。

5. 解决 Pintos 操作系统中进程终止时缺少终端提示的问题：锻炼调试和改进操作系统代码的能力。

6. 处理 Pintos 操作系统中运行文件禁止写操作的问题：学会如何管理文件权限和操作。

具体需要实现的目标有：

1、完成参数传递和部分系统调用（exit 和 write），使得 make check 通过 args 相关的 5 个测试。

2、完成实验报告并提交。无需提交代码文件，只需要提交报告的 pdf 即可。在报告中展示代码中需要修改的地方，解释修改的原因，和运行结果。

3、下次作业预告：下次课讲解部分其他系统调用的实现，全部系统调用的实现可作为下次作业提交，本次只需完成 exit 和 write 即可。

二、内容与设计思想

一、实验内容

1. 参数传递

2. 系统调用实现（部分）

3. 进程退出信息打印

二、设计思想

1. 参数传递

1.1 问题描述

在 Pintos 操作系统中，用户程序启动时需要将命令和参数传递给用户程序。默认情况下，process_execute 函数接收到的文件名（file_name）包含了命令和参数。我们需要将命令和参数分离，并将参数正确地压入栈中。

1.2 设计思路

• 字符串解析：使用 strtok_r 函数将 file_name 分解成命令和参数。

• 栈布局：将参数按 C 语言标准的栈布局压入栈中。

• 线程同步：使用信号量确保 start_process 线程在 process_execute 线程之前结束。

2. 系统调用实现

2.1 问题描述

Pintos 操作系统默认没有实现系统调用。我们需要实现 exit 和 write 系统调用，使用户程序可以退出并打印到控制台。

2.2 设计思路

• 系统调用编号：定义系统调用编号，以便在中断处理时识别不同的系统调用。

• 系统调用处理函数：为每个系统调用实现处理函数。

• 中断处理：在中断处理函数中根据系统调用编号调用相应的处理函数。

3. 进程退出信息打印

3.1 问题描述

当进程退出时，Pintos 操作系统没有提供终端提示信息。我们需要在进程退出时打印相关信息。

3.2 设计思路

• 修改进程退出函数：在 process_exit 函数中添加打印语句，输出进程退出的相关信息。

• 格式化输出：确保输出格式符合要求。

4. 运行结果

1. 参数传递测试

• 测试用例：/bin/ls -l foo bar

• 测试结果：成功解析命令和参数，并正确压入栈中。make check 通过相关测试。

2. 系统调用测试

• 测试用例：调用 exit 和 write 系统调用。

• 测试结果：exit 系统调用成功退出进程，write 系统调用成功打印到控制台。make check 通过相关测试。

3. 进程退出信息打印

• 测试用例：运行一个进程并退出。

• 测试结果：成功打印进程退出信息，格式符合要求。

通过本次实验，我们深入理解了 Pintos 操作系统的内核结构和功能流程，掌握了参数传递、系统调用和进程退出信息打印的实现方法。实验不仅提高了我们的编程能力，还增强了我们对操作系统内核的理解。通过实现扩展任务，我们进一步完善了 Pintos 操作系统的功能，使其更加符合实际应用需求。

三、使用环境

本次实验使用 Visual Studio Code (VSCode) 作为开发环境。VSCode 是微软研发的一款免费 IDE，适用于 Windows 和 macOS 系统，尤其适合进行系统开发。相比于 IntelliJ 系列的 IDE，VSCode 不仅免费，而且在远程开发方面有较好的支持。

四、实验过程

1. 创建磁盘

在 Pintos 中，首先需要创建一个磁盘来存储用户程序。这个步骤通常在实验环境中已经配置好，但在实际操作中，可以通过在 userprog/build 目录下执行以下命令创建一个磁盘文件：

```
pintos-mkdisk filesys.dsk --filesys-size=2
pintos -p ../../examples/echo.c -a echo -- -f -q run 'echo x'
```

2. 参数传递

参数传递是 Pintos 用户程序执行的关键步骤之一。默认情况下，process_execute()函数接收的 file_name 包含了命令和参数，我们需要将其分离，使得 file_name 只包含命令，而参数则通过栈传递。

2.1 分离命令和参数

首先，我们需要在 process_execute()函数中使用 strtok_r()函数来分离命令和参数。strtok_r()函数可以将字符串按指定的分隔符分割成多个部分。

<process.c>中的 process_execute 提供了 file_name ，其包括了命令和参数字符串。我们要将 file_name 拆分以分割命令和参数。然后我们把这个命令当作新进程的名字，并把参数传递给函数 start_process , load 和 setup_stack 。然后，分析 process_execute 与 start_process 原来的代码。我们可以知道 process_execute 创建线程之后，用户程序不会立刻执行，他会进入 start_process ，然后这个函数又会调用 load ，将会为用户程序分配内存。所以设置栈的时间是在 load 之后。为了设置堆栈，我们首先要对参数和命令的名字进行 memcpy 处理，并保存他们的地址以备将来使用。然后添加对齐方式并且之前存储的 argv 地址还要确保 argv[argc] 是一个空指针。接下来添加 argv , argc 的地址，最后添加 return 的地址。原代码复制文件名：fn_copy = palloc_get_page(0);：分配一页内存来复制 file_name。strlcpy(fn_copy, file_name, PGSIZE);：将 file_name 复制到 fn_copy 中。如果分配内存失败，返回 TID_ERROR。

修改后的 process_execute()函数如下：

```c
/** Starts a new thread running a user program loaded from
   FILENAME.  The new thread may be scheduled (and may even exit)
   before process_execute() returns.  Returns the new process's
   thread id, or TID_ERROR if the thread cannot be created. */
tid_t
process_execute (const char *file_name)
{
  char *fn_copy;
  tid_t tid;

  /* Make a copy of FILE_NAME.
     Otherwise there's a race between the caller and load(). */
  fn_copy = palloc_get_page (0);
  if (fn_copy == NULL)
    return TID_ERROR;
  strlcpy (fn_copy, file_name, PGSIZE);

  // Extract real filename, zzw231220
  char *fn_real;
  fn_real = palloc_get_page (0);
  if (fn_real == NULL)
    return TID_ERROR;
  strlcpy (fn_real, file_name, PGSIZE) ;
  char *save_ptr;
  fn_real = strtok_r(fn_real, " ", &save_ptr);

  /* Create a new thread to execute FILE NAME. */
  tid = thread_create (fn_real, PRI_DEFAULT, start_process, fn_copy);
  palloc_free_page (fn_real);

  // /* Create a new thread to execute FILE_NAME. */
  // tid = thread_create (file_name, PRI_DEFAULT, start_process, fn_copy);
  if (tid == TID_ERROR) {
        palloc_free_page (fn_copy);
        return tid;
  }

  sema_down(&thread_current()->sema);
  return tid;
}
```

修改后代码解释：

复制文件名：

fn_copy = palloc_get_page(0);：分配一页内存来复制 file_name。

strlcpy(fn_copy, file_name, PGSIZE);：将 file_name 复制到 fn_copy 中。

如果分配内存失败，返回 TID_ERROR。

提取实际文件名：

fn_real = palloc_get_page(0);：分配一页内存来提取实际文件名。

strlcpy(fn_real, file_name, PGSIZE);：将 file_name 复制到 fn_real 中。

fn_real = strtok_r(fn_real, " ", &save_ptr);：使用 strtok_r 函数从 fn_real 中提取实际文件名（假设 file_name 可能包含额外的参数）。

如果分配内存失败，返回 TID_ERROR。

创建新线程：

tid = thread_create(fn_real, PRI_DEFAULT, start_process, fn_copy);：创建一个新线程来执行 fn_real，并传递 fn_copy 作为参数。

释放 fn_real。

如果创建线程失败，释放 fn_copy 并返回 TID_ERROR。

同步：

sema_down(&thread_current()->sema);：等待新线程完成某些操作。

修改原因：

提取实际文件名：

原始代码中直接使用 file_name 作为新线程的文件名，但 file_name 可能包含额外的参数（例如命令行参数）。修改后的代码使用 strtok_r 函数从 file_name 中提取实际文件名，确保新线程只执行实际的文件。

代码健壮性：

通过提取实际文件名，代码更加健壮，可以处理包含额外参数的 file_name，避免了潜在的错误。

通过这些修改，process_execute 函数更加健壮和可靠，能够更好地处理包含额外参数的文件名。

2.2 参数压栈

在 start_process()函数中，我们需要将分离出的参数按要求压入栈中。栈的布局要求参数从最后一个参数开始压栈。在修改前的 start_process 函数中，主要做了以下几件事：在修改前的代码中，start_process 函数的主要目的是加载一个用户进程并启动它运行。以下是关键步骤的解释：1. 初始化中断帧 (intr_frame)：memset (&if_, 0, sizeof if_);if_.gs = if_.fs = if_.es = if_.ds = if_.ss = SEL_UDSEG;if_.cs = SEL_UCSEG;if_.eflags = FLAG_IF | FLAG_MBS;这部分代码初始化了一个中断帧，设置了一些段选择器和标志位。2. 加载可执行文件：success = load (file_name, &if_.eip, &if_.esp);这里调用了 load 函数来加载用户进程的可执行文件，并将指令指针 (eip) 和栈指针 (esp) 设置为加载后的值。3. 处理加载失败：if (!success) thread_exit ();如果加载失败，直接退出线程。4. 启动用户进程：asm volatile ("movl %0, %%esp; jmp intr_exit" : : "g" (&if_) : "memory");通过汇编代码将栈指针 (esp) 指向中断帧，并跳转到 intr_exit 函数，模拟从中断返回的过程，从而启动用户进程。

修改后的 start_process()函数如下：

```
/** A thread function that loads a user process and starts it
   running. */
static void
start_process (void *file_name_)
{
  char *file_name = file_name_;
  struct intr_frame if_;
  bool success;

  /* Initialize interrupt frame and load executable. */
  memset (&if_, 0, sizeof if_);
  if_.gs = if_.fs = if_.es = if_.ds = if_.ss = SEL_UDSEG;
  if_.cs = SEL_UCSEG;
  if_.eflags = FLAG_IF | FLAG_MBS;
  char *fn_copy=palloc_get_page (0);
  strlcpy(fn_copy, file_name, strlen(file_name)+1);
  char *save_ptr;
  // real file_name, zzw231220
  file_name=strtok_r(file_name," ",&save_ptr);
  success = load (file_name, &if_.eip, &if_.esp);

  /* If load failed, quit. */
  palloc_free_page (file_name);
  if (!success)
    thread_exit ();

  // bulid stack, zzw231220
  int argc = 0; // number of args
  int argv[32];
  char *token;

  // 1. push args into stack, include file_name
  for(token=strtok_r(fn_copy," ", &save_ptr); token != NULL;
    token=strtok_r(NULL," ",&save_ptr) ) {
    if_.esp -= (strlen(token)+1); // esp give space to token(arg)
```

```
// 1. push args into stack, include file_name
for(token=strtok_r(fn_copy," ", &save_ptr); token != NULL;
  token=strtok_r(NULL," ",&save_ptr) ) {
  if_.esp -= (strlen(token)+1); // esp give space to token(arg)
  memcpy(if_.esp, token, strlen(token)+1); // copy arg to stack
  argv[argc]=(int) if_.esp; // record address of arg
  argc++;
}

//printf(">> argnum: %d.\n",argc);

// 2. push address of args into stack
void **esp = &if_.esp;
*esp = (int)*esp & 0xfffffffc;
*esp -= 4;
*(int *)*esp = 0;
for(int i = argc - 1; i>=0; i -- ){
  *esp -= 4;
  *(int *)*esp = argv[i];
}
*esp -= 4;
*(int *)*esp = (int)*esp + 4; // push address of argv
*esp -= 4;
*(int *)*esp = argc; // push argc
*esp -= 4;
*(int *)*esp = 0; // push return address

/* Start the user process by simulating a return from an
   interrupt, implemented by intr_exit (in
   threads/intr-stubs.S).  Because intr_exit takes all of its
   arguments on the stack in the form of a `struct intr_frame',
   we just point the stack pointer (%esp) to our stack frame
   and jump to it. */
asm volatile ("movl %0, %%esp; jmp intr_exit" : : "g" (&if_) :
"memory");
NOT_REACHED ();

sema_up(&thread_current()->sema);
}
```

修改后代码解释：

修改后的代码主要解决了参数传递的问题，特别是如何将参数正确地传递给用户进程。以下是修改后的关键步骤：

1. 复制文件名：

```
char *fn_copy = palloc_get_page (0);strlcpy(fn_copy, file_name, strlen(file_name)+1);
```

这部分代码将传入的 file_name 复制到一个新的内存页中，以便后续的参数解析不会影响原始字符串。

2. 解析命令行参数：

```
file_name = strtok_r(file_name, " ", &save_ptr);
```

使用 strtok_r 函数解析命令行参数，将 file_name 分解成多个参数。

3. 加载可执行文件：

```
success = load (file_name, &if_.eip, &if_.esp);
```

与修改前相同，调用 load 函数加载可执行文件。

4. 处理加载失败：

```
if (!success)
  thread_exit ();
```

如果加载失败，直接退出线程。

5. 构建栈帧：

```
int argc = 0; int argv[32];
```

初始化 argc 和 argv 数组，用于记录参数的数量和地址。

6. 将参数压入栈：

```
for(token = strtok_r(fn_copy, " ", &save_ptr); token != NULL; token = strtok_r(NULL, " ", &save_ptr)) {
  if_.esp -= (strlen(token)+1);
  memcpy(if_.esp, token, strlen(token)+1);
  argv[argc] = (int) if_.esp;
  argc++;}
```

遍历解析出的参数，将每个参数压入栈中，并记录参数的地址。

7. 将参数地址压入栈：

```
void **esp = &if_.esp;*esp = (int)*esp & 0xfffffffc;*esp -= 4;*(int *)*esp = 0;for(int i = argc - 1; i >= 0; i--) {
  *esp -= 4;
  *(int *)*esp = argv[i];}*esp -= 4;*(int *)*esp = (int)*esp + 4; *esp -= 4;*(int *)*esp = argc; *esp -= 4;*(int *)*esp = 0;
```

这部分代码将参数的地址（即 argv）压入栈中，并设置 argc 和返回地址。

8. 启动用户进程：

```
asm volatile ("movl %0, %%esp; jmp intr_exit" : : "g" (&if_) : "memory");
```

与修改前相同，通过汇编代码将栈指针 (esp) 指向中断帧，并跳转到 intr_exit 函数，模拟从中断返回的过程，从而启动用户进程。

修改原因：

修改的原因主要是为了解决参数传递的问题。在修改前的代码中，start_process 函数并没有正确处理用户进程启动时的命令行参数传递。具体问题包括：

1. 参数解析缺失：原始代码没有解析命令行参数，直接将 file_name 传递给 load 函数，这可能导致无法正确处理多个参数的情况。

2. 栈帧构建不完整：原始代码没有构建完整的栈帧，特别是没有将参数和参数地址正

确地压入栈中，这会导致用户进程启动时无法接收到正确的参数。
通过修改后的代码，解决了这些问题：

1. 参数解析：使用 strtok_r 函数解析命令行参数，确保能够处理多个参数。
2. 栈帧构建：正确地将参数和参数地址压入栈中，确保用户进程能够接收到正确的参数。
3. 参数传递：通过构建正确的栈帧，用户进程在启动时可以从栈中获取传入的参数，从而正确执行。

修改后的代码通过增加参数解析和栈帧构建的步骤，确保了用户进程在启动时能够正确接收和使用传入的参数。这种修改是为了解决原始代码中参数传递不完整的问题，从而提高了系统的稳定性和功能性。

2.3 线程同步

为了避免 process_execute()在线程 start_process()结束前就结束，我们需要使用信号量进行同步。

修改后的对应代码如下：

```
#include <debug.h>
#include <list.h>
#include <stdint.h>
#include "threads/synch.h"
```

```
#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;
    struct semaphore sema;
    struct thread* parent;
#endif
```

```
#ifdef USERPROG
  sema_init(&t->sema, 0);
  if(t == initial_thread) t->parent=NULL;
  else t->parent=thread_current();
#endif
```

```
    }

    sema_down(&thread_current()->sema);
    return tid;
}
```

```
    asm volatile ( movl %0, %%esp; jmp intr
    "memory");
    NOT_REACHED ();

    sema_up(&thread_current()->sema);
}
```

修改原因：

信号量用于确保 start_process()在 process_execute()结束前完成：为了确保 start_process 在线程 process_execute 结束之前完成，我们需要使用信号量进行同步。在 thread.c 中，我们初始化信号量，并在 start_process 结束前调用 sema_up，在 process_execute 结束前调用 sema_down。

3. 系统调用

系统调用是用户程序与操作系统内核通信的主要方式。在本实验中，我们需要实现 exit 和 write 系统调用。在修改前的代码中，系统调用的处理逻辑非常简单且不完整。以下是关键部分的分析：1. 系统调用初始化 (syscall_init):使用 intr_register_int 注册了一个中断处理函数 syscall_handler，该中断处理函数会在发生中断时被调用。2. 系统调用处理函数 (syscall_handler):该函数只是简单地打印 “system call!”，然后调用 thread_exit() 退出当前线程。这意味着任何系统调用都会导致线程退出，而没有任何实际的系统调用处理逻辑。

修改后的对应代码如下：

src > userprog > C syscall.c > syscall_init(void)

```c
#include "userprog/syscall.h"
#include <stdio.h>
#include <syscall-nr.h>
#include "threads/interrupt.h"
#include "threads/thread.h"

#define max_syscall 20
static void syscall_handler (struct intr_frame *);

static void (*syscalls[max_syscall]) (struct intr_frame *);

void sys_exit(struct intr_frame* f);
void sys_write(struct intr_frame* f);

void
syscall_init (void)
{
  intr_register_int (0x30, 3, INTR_ON, syscall_handler, "syscall");
  syscalls[SYS_EXIT]=&sys_exit;
  syscalls[SYS_WRITE]=&sys_write;
}

static void
syscall_handler (struct intr_frame *f UNUSED)
{
  int type =* (int*)f->esp;
  if(type <= 0||type>=max_syscall) {
  printf("invalid system call!\n");
  thread_exit();
  }
  if(type == SYS_EXIT || type == SYS_WRITE)
    syscalls [type](f);
  else{
    printf ("system call!\n");
    thread_exit ();
  }
}
```

```
{
  int type =* (int*)f->esp;
  if(type <= 0||type>=max_syscall) {
  printf("invalid system call!\n");
  thread_exit();
  }
  if(type == SYS_EXIT || type == SYS_WRITE)
    syscalls [type](f);
  else{
    printf ("system call!\n");
    thread_exit ();
  }
}

// prototype: void exit (int status) NO_RETURN;
void sys_exit(struct intr_frame* f) {
  uint32_t *user_ptr=f->esp;
  *user_ptr++;
  thread_current()->st_exit = *user_ptr;
  thread_exit();
}

// prototype: int write (int fd, const void *buffer, unsigned length)
void sys_write(struct intr_frame* f) {
  uint32_t *user_ptr=f->esp;
  *user_ptr++;
  printf("user_ptr:%x, fesp:%x\n", user_ptr, f->esp);
  int tmp =* user_ptr;
  const char* buffer=(const char *)*(user_ptr+1);
  unsigned size =* (user_ptr+2);
  if(tmp == 1){
    // write to console
    putbuf (buffer,size);
    f->eax=size;
  }
}
```

修改后代码解释：

在修改后的代码中，系统调用的处理逻辑得到了显著的改进和扩展。以下是关键部分的分析：

1. 系统调用初始化 (syscall_init):

   仍然使用 intr_register_int 注册了中断处理函数 syscall_handler。

   增加了 syscalls 数组，用于存储不同系统调用的处理函数指针。例如，SYS_EXIT 和 SYS_WRITE 系统调用的处理函数分别被注册为 sys_exit 和 sys_write。

2. 系统调用处理函数 (syscall_handler):

   从中断帧 f 中获取系统调用类型（type）。

   检查 type 是否在有效范围内（1 到 max_syscall 之间）。如果不在范围内，打印 “invalid system call!” 并退出线程。

   如果在有效范围内，则根据 type 调用相应的系统调用处理函数。

3. 具体系统调用处理函数:

   sys_exit:

   从用户栈指针 esp 中提取退出状态，并将其存储在当前线程的退出状态字段 st_exit 中。

调用 thread_exit() 退出当前线程。

sys_write:

从用户栈指针 esp 中提取文件描述符、缓冲区指针和写入长度。

如果文件描述符为 1（通常表示标准输出），则调用 putbuf 将缓冲区的内容输出到控制台。

将写入的字节数返回给用户。

修改原因：

syscall_init()函数用于将系统调用编号与处理函数关联。

1.引入系统调用分发机制:

修改前的代码没有区分不同的系统调用类型，所有系统调用都被视为相同，导致无法处理具体的系统调用请求。

通过引入 syscalls 数组和系统调用类型检查，修改后的代码可以根据不同的系统调用类型分发到相应的处理函数。

2.实现具体系统调用功能:

修改前的代码没有实现任何具体的系统调用功能，所有系统调用都只是简单地退出线程。

修改后的代码实现了 SYS_EXIT 和 SYS_WRITE 两个具体的系统调用功能，使得用户程序可以通过系统调用实现退出和输出到控制台的功能。

3.增强代码的健壮性:

修改后的代码增加了对系统调用类型的检查，防止无效的系统调用类型导致系统崩溃。

修改前的代码没有进行任何有效性检查，可能导致系统在接收到无效系统调用时直接崩溃。

通过这些修改，系统调用机制变得更加灵活、健壮和功能丰富，能够处理多种系统调用请求，并提供具体的系统调用功能。

4. 打印进程退出信息

在进程退出时，我们需要打印进程的退出信息。在 process.c 中修改 process_exit()函数：

修改后的对应代码如下：

```c
/** Free the current process's resources. */
void
process_exit (void)
{
  struct thread *cur = thread_current ();
  uint32_t *pd;

  // print process termination message, ptm, zzw231220
  printf("%s: exit(%d)\n",thread_name(), cur->st_exit);
  /* Destroy the current process's page directory and switch back
     to the kernel-only page directory. */
  pd = cur->pagedir;
  if (pd != NULL)
    {
      /* Correct ordering here is crucial.  We must set
         cur->pagedir to NULL before switching page directories,
         so that a timer interrupt can't switch back to the
         process page directory.  We must activate the base page
         directory before destroying the process's page
         directory, or our active page directory will be one
         that's been freed (and cleared). */
      cur->pagedir = NULL;
      pagedir_activate (NULL);
      pagedir_destroy (pd);
    }
}
```

修改原因：

打印进程的退出状态，便于调试和查看。打印进程终止消息 printf("%s: exit(%d)\n", thread_name(), cur->st_exit);这里打印出当前进程的名称和退出状态码。这有助于调试和了解进程的退出情况。

5. make check 结果展示

```
(Process exit codes are excluded for matching purposes. )
pass tests/userprog/args-none
pass tests/userprog/args-single
pass tests/userprog/args-multiple
pass tests/userprog/args-many
pass tests/userprog/args-dbl-space
FAIL tests/userprog/sc-bad-sp
FAIL tests/userprog/sc-bad-arg
pass tests/userprog/sc-boundary
pass tests/userprog/sc-boundary-2
FAIL tests/userprog/sc-boundary-3
pass tests/userprog/halt
pass tests/userprog/exit
```

五、总结

在本次实验中，我们深入学习和实践了操作系统的系统调用和进程管理机制。通过这次实验，我们对操作系统的核心概念有了更深刻的理解，并且提高了在实际操作中的动手能力。

系统调用是用户程序与操作系统之间的接口，它允许用户程序请求操作系统提供的服务。在本次实验中，我们主要实现了两个系统调用：exit 和 write。

1. exit 系统调用：实现了 sys_exit 函数，该函数负责处理进程的退出。当用户程序调用 exit 时，系统调用会捕获这个请求，并通过 sys_exit 函数来执行相应的退出操作，包括释放进程的页目录和清理相关资源。

2. write 系统调用：实现了 sys_write 函数，该函数负责将数据从用户空间写入到指定的文件描述符中。在本次实验中，我们主要处理了标准输出（文件描述符 1），通过调用 putbuf 函数将用户数据输出到控制台。

通过这些系统调用的实现，我们理解了用户程序如何与操作系统内核进行交互，以及内核如何处理这些请求并提供服务。

进程管理是操作系统的核心功能之一，负责创建、调度和销毁进程。在本次实验中，我们主要关注了进程的退出机制。

1. 进程退出：在 process_exit 函数中，我们实现了进程退出时的清理工作，包括销毁进程的页目录和释放相关资源。通过打印进程的终止消息，我们可以清楚地看到进程的退出状态和名称，这有助于调试和监控进程的运行情况。

2. 多进程管理：通过模拟多个进程的创建和退出，我们观察了操作系统的进程调度和管理机制。理解了进程间的切换和资源的分配方式，以及内核如何确保进程之间的隔离和安全性。

通过本次实验，我们深入理解了系统调用的原理和实现方法，包括系统调用号的识别、参数的传递和返回值的处理。掌握了进程的创建、调度和退出等基本操作，以及内核如何管理进程的生命周期。学习了页目录的管理和释放，理解了虚拟内存的原理和实现。在实验过程中，通过打印调试信息和观察进程的行为，提高了调试和测试的能力。

总之，通过本次实验，我们对操作系统的核心概念和实现机制有了更深入的理解，并且在实际操作中提高了编程和调试的能力。这为今后的学习和研究打下了坚实的基础。

六、附录

GitHub 地址：Redamancy-Xun/Operating-System-Practice: 操作系统实践 (github.com)